



UT5: Clases, Objetos y Herencia Avanzado

Resumen completo de Programación Orientada a Objetos en Java



Visión Global

Una aplicación real combina varios principios fundamentales de la POO:



Clases

Plantillas que definen la estructura y comportamiento de los objetos.



Objetos

Instancias concretas de una clase con estado propio.



Herencia

Permite reutilizar y especializar comportamiento de otras clases.



Polimorfismo

Muchas formas para una misma operación (sobrescritura/sobrecarga).



Interfaces

Contratos que definen capacidades ("qué" debe hacer).



Abstract

Base común con implementación parcial.



Enum

Conjunto finito de valores semánticos.



Composición

Un objeto contiene otros para delegar responsabilidades.



Enum para Roles

Un **enum** es un conjunto cerrado de valores semánticos, útil para evitar "magic strings".

```
enum Rol { ADMIN, PROFESOR, ALUMNO }
```



¿Cuándo usar Enum?

Cuando tienes un conjunto fijo de opciones que no cambiará, como estados, roles, tipos, etc.



Interfaces (Contratos)

Las interfaces definen "**qué**" debe hacerse, no "**cómo**". Permiten múltiples capacidades.

```
// Interfaz para objetos identificables interface Identificable { String getId(); }  
// Interfaz para objetos que pueden calcular pagos interface Pagable { double  
calcularPago(); } // Capacidades específicas de animales interface Volador { void  
volar(); } interface Nadador { void nadar(); }
```



Encapsulación + Patrón Builder

✓ Principios de Encapsulación

- Mantén los atributos **privados**
- Expón acceso controlado mediante **getters/setters**
- Valida datos en setters o constructores

```
class Persona implements Identificable { // Atributos privados (encapsulación)
    private final String id; // Inmutable una vez creado
    private String nombre;
    private int edad;
    private Rol rol; // Constructor con validaciones
    public Persona(String nombre, int edad) {
        this.id = "P-" + System.nanoTime();
        setNombre(nombre); // Reutiliza validaciones
        setEdad(edad);
        this.rol = Rol.ALUMNO;
    } // Setter con validación
    public void setEdad(int edad) {
        if (edad < 0) throw new IllegalArgumentException("Edad no puede ser negativa");
        this.edad = edad;
    }
}
```



Patrón Builder

Facilita la construcción de objetos con muchos parámetros usando llamadas encadenadas:

```
// Clase Builder interna
public static class Builder {
    private String id = "P-" + System.nanoTime();
    private String nombre;
    private int edad;
    private Rol rol = Rol.ALUMNO;
    public Builder nombre(String nombre) {
        this.nombre = nombre;
        return this;
    }
    public Builder edad(int edad) {
        this.edad = edad;
        return this;
    }
    public Builder rol(Rol rol) {
        this.rol = rol;
        return this;
    }
    public Persona build() {
        return new Persona(id, nombre, edad, rol);
    }
} // Uso del Builder:
Persona p = new Persona.Builder()
    .nombre("Laura")
    .edad(28)
    .rol(Rol.ADMIN)
    .build();
```



Herencia y Polimorfismo

La subclase **especializa** a la superclase. Sobrescribe métodos para comportamiento específico.

```
class Estudiante extends Persona implements Pagable { private String cursoActual;
private double cuotaMensual; public Estudiante(String nombre, int edad, String
curso, double cuota) { super(nombre, edad); // Llama al constructor padre
this.cursoActual = curso; this.cuotaMensual = cuota; setRol(Rol.ALUMNO); }
@Override public double calcularPago() { return cuotaMensual; } public void
estudiar() { System.out.println("■ Estudiando en: " + cursoActual); } } class
Empleado extends Persona implements Pagable { private double salarioBase; private
double bonus; @Override public double calcularPago() { return salarioBase + bonus;
} }
```



Clases Abstractas

Tienen implementación parcial y métodos abstractos que las subclases deben completar.

```
abstract class Animal implements Identificable { private final String id = "A-" +
System.nanoTime(); protected String nombre; // protected permite acceso en
subclases public Animal(String nombre) { this.nombre = nombre; } public abstract
void hacerSonido(); // Método abstracto // Hook method (opcional) que subclases
pueden usar protected void onAfterSound() { } } class Perro extends Animal { public
Perro(String nombre) { super(nombre); } @Override public void hacerSonido() {
System.out.println(nombre + " ladra: Guau!"); onAfterSound(); } }
```

Característica	Interface	Clase Abstracta
Implementación	Solo firmas (por defecto)	Parcial (métodos concretos + abstractos)
Herencia	Múltiple (implements)	Simple (extends)
Atributos	Solo constantes (static final)	Cualquier tipo
Uso	Definir capacidades	Base común con lógica compartida



Composición

Un objeto **contiene otros** objetos para delegar responsabilidades (ej: Curso contiene Personas).

```
class Curso { private final String nombre; private final List<Persona>
participantes = new ArrayList<>(); public void agregarParticipante(Persona p) {
participantes.add(p); } public double ingresosTotales() { double total = 0; for
(Persona p : participantes) { if (p instanceof Pagable) { total += ((Pagable)
p).calcularPago(); } } return total; } }
```



Overloading vs Overriding

Concepto	Descripción	Ejemplo
OVERLOADING (Sobrecarga)	Mismo nombre, distinta firma (en la misma clase)	<pre>imprimir(String s) imprimir(int n)</pre>
OVERRIDING (Sobrescritura)	Redefinir método de la superclase (polimorfismo)	<pre>@Override hacerSonido()</pre>

```
// Ejemplo de Overloading
class Printer {
    public static void imprimir(String s) {
        System.out.println("[String] " + s);
    }
    public static void imprimir(int n) {
        System.out.println("[int] " + n);
    }
    public static void imprimir(Persona p) {
        System.out.println("[Persona] " + p);
    }
}

// Ejemplo de Overriding
class Circulo extends Figura {
    @Override double area() {
        return Math.PI * r * r;
    }
    // Sobrescribe
    double area(double escala) {
        return area() * escala;
    }
    // Sobrecarga
}
```



Genéricos

Evitan casts inseguros y mejoran la reutilización del código.

```
// Clase genérica simple para envolver cualquier tipo de valor
class Box<T> {
    private T valor;
    public Box(T valor) {
        this.valor = valor;
    }
    public T get() {
        return valor;
    }
    public void set(T valor) {
        this.valor = valor;
    }
}

// Uso:
Box<String> boxTexto = new Box<>("Mensaje");
Box<Integer> boxNumero = new Box<>(42);
```



Tareas Avanzadas para el Alumno

1. Crear una nueva subclase de Animal llamada "Leon" con método extra `rugir()`.
2. Añadir una interfaz propia "Corredor" con método `correr()` e implementarla en Perro y Leon.
3. Extender el Builder de Persona para asignar una lista de "habilidades".
4. Crear una clase genérica `Par<A,B>` para emparejar Estudiante con Curso.
5. Agregar validación: impedir que `cuotaMensual` en Estudiante sea ≤ 0 .
6. Crear método estático en Estadísticas para obtener el máximo pago entre Pagables.
7. Implementar subclase `EmpleadoHoras` ($\text{pago} = \text{horas} \times \text{tarifa}$).
8. Añadir sobrecarga en `Printer` para `List<?>`.
9. Crear clase `RegistroActividad` que reciba `Identificable` y guarde log.
10. Refactorizar `Curso` para usar generics: `Curso<T extends Persona>`.



Resumen Visual

✓ Checklist de Conceptos Clave

- ✓ **Encapsulación:** Atributos privados + getters/setters con validación
- ✓ **Herencia:** `extends` para especializar clases
- ✓ **Polimorfismo:** `@Override` para redefinir comportamiento
- ✓ **Interfaces:** `implements` para capacidades múltiples
- ✓ **Abstract:** Base parcial con métodos obligatorios
- ✓ **Composición:** Objetos que contienen otros objetos
- ✓ **Genéricos:** `Clase<T>` para reutilización segura
- ✓ **Builder:** Construcción fluida de objetos complejos
- ✓ **Enum:** Valores fijos y semánticos

 UT5 - Clases, Objetos y Herencia Avanzado | Java POO

Generado automáticamente - Para imprimir: Ctrl+P → Guardar como PDF